AI - Lab

5. Object Detection



Hochschule Karlsruhe
Technik und Wirtschaft
UNIVERSITY OF APPLIED SCIENCES

Dr. Fabian Sigloch

WS 26/27

Week 4 follow-up: imbalanced classes

Hochschule Karlsruhe University of Applied Sciences HKA

Illustrative classification confusion matrix

(rows: true class; columns: predicted class)

	Predicted			**Support**
True	A	B	C	
A (common)	**90**	8	2	100
B	10	**30**	10	50
C (rare)	5	10	**10**	25
Total	105	48	22	175

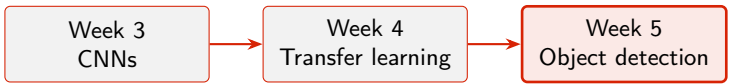
- Overall top-1: $130/175 = 74.3\%$.
- Per-class top-1: 90%, 60%, 40%.
- Mean per-class: $(90 + 60 + 40)/3 = 63.3\%$.
- Majority baseline (always predict A): $100/175 = 57.1\%$.

Why report more than top-1?

Overall top-1 and top-3 weight each image equally, so common classes dominate. Macro scores weight classes equally. Flowers102's test split is imbalanced; choose the majority class from training data for a fair baseline.

Illustrative numbers only; assume A was chosen using training data. These are not Flowers102 results.

Where we are in the lab

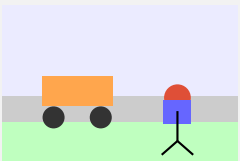


- Week 3 classified one whole image.
- Week 4 reused a pre-trained visual backbone for a new classification task.
- Today, one image may contain **several objects**; we predict their classes *and* locations.

Classification is not enough

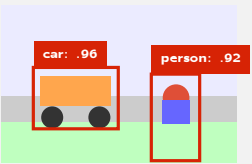
Hochschule Karlsruhe
University of
Applied Sciences
HKA

Image classification



Prediction: “street scene”

Object detection



Predictions: car, person *and locations*

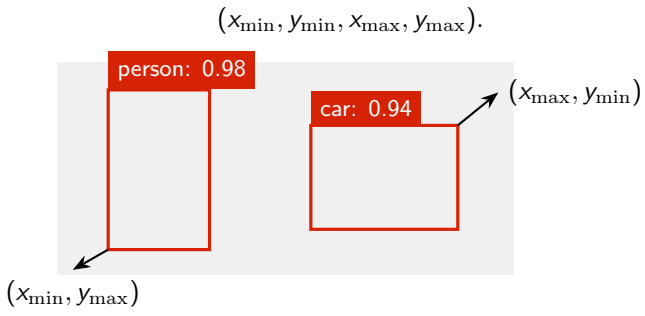
The object-detection task

Hochschule Karlsruhe University of Applied Sciences HKA

For every detected object, a detector predicts:

1. a **class label**, e.g. `person` or `car`;
2. a **confidence score**;
3. a **bounding box** in pixel coordinates:

$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.



What changes compared with classification?

Hochschule Karlsruhe University of Applied Sciences HKA

Classification

- One image → one class vector.
- Fixed output shape: `(N, C)`.
- One target label per image.

Detection

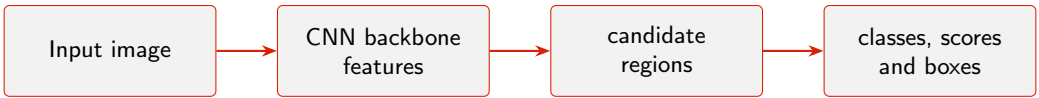
- One image → zero, one, or many objects.
- Variable number of output boxes.
- A target is a dictionary of boxes and labels.

Detection target for one image

```
{'boxes': Tensor(N, 4), 'labels': Tensor(N)}
```

Here, N is the number of annotated objects in this particular image, not the batch size.

From a CNN classifier to a detector



- The **backbone** is a CNN, often a ResNet, and extracts feature maps.
- A **feature pyramid** provides features at several spatial scales: small and large objects need different resolutions.
- The **detection head** proposes regions, classifies them, and adjusts their boxes.

Faster R-CNN: a two-stage detector

Hochschule Karlsruhe University of Applied Sciences HKA

1. A backbone + feature pyramid converts the image to feature maps.
2. The **Region Proposal Network** (RPN) marks regions likely to contain any object.
3. The second stage classifies each proposed region and refines its bounding box.

Why two stages?

The first stage answers "where might an object be?" The second answers "what is it, and where exactly is it?"

Lab choice

We use a pre-trained detector. We do *not* implement proposals or box regression from scratch.

Alternatives: one-stage detectors

Hochschule Karlsruhe
University of
Applied Sciences
HKA

Two-stage: Faster R-CNN

- First proposes regions, then classifies them.
- Often a strong accuracy-oriented baseline.
- More computation and implementation machinery.

One-stage: YOLO, SSD, RetinaNet

- Predicts classes and boxes directly across image locations.
- Often designed for high inference speed.
- Has its own matching and loss-design choices.

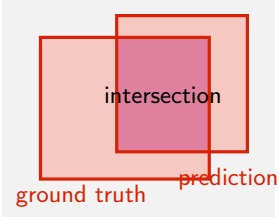
The conceptual output is identical: a set of labelled boxes with scores.

Intersection over Union (IoU)

Hochschule Karlsruhe University of Applied Sciences HKA

$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}.$$

- IoU $= 1$: boxes are identical.
- IoU $= 0$: boxes do not overlap.
- A prediction needs the right class *and* enough overlap to count as a match.



From matches to precision, recall, and mAP

Hochschule Karlsruhe University of Applied Sciences HKA

At a chosen IoU threshold (often 0.5), sort predictions by confidence and match each ground-truth object at most once.

$$\text{precision} = \frac{TP}{TP + FP}$$

Of the predicted objects, how many are correct?

$$\text{recall} = \frac{TP}{TP + FN}$$

Of the real objects, how many were found?

- Raising a score threshold usually increases precision but reduces recall.
- **Average Precision** (AP) summarises the precision–recall curve for one class.
- **mAP** is the mean AP across classes and, in COCO-style evaluation, often across several IoU thresholds.

Confidence threshold and duplicate boxes

Hochschule Karlsruhe University of Applied Sciences HKA

A detector can return many overlapping proposals for the same object.

Score threshold

- Discard predictions below a chosen confidence score.
- Low threshold: more detections, more false positives.
- High threshold: fewer false positives, more missed objects.

Non-maximum suppression (NMS)

- Keep the highest-scoring box.
- Remove strongly overlapping lower-score boxes of the same class.
- Prevents several boxes being reported for one object.

Transfer learning, again

Hochschule Karlsruhe University of Applied Sciences HKA

- A detection model can be pre-trained on **COCO**: its weights already recognise common objects and their boxes.
- For a new task, replace the detection classifier head so it has the target number of classes *plus background*.
- Fine-tuning adapts the detector to the new labels and image domain; a small annotated dataset is enough for this lab because it starts from strong weights.

Detection labels

Classification used labels $0, \ldots, C-1$. In torchvision detection, label 0 is reserved for **background**; a one-class pedestrian task therefore uses `num_classes=2` and pedestrian label `1`.

Inference with a COCO-pre-trained detector

Hochschule Karlsruhe University of Applied Sciences HKA

```python
from torchvision.models.detection import (
    fasterrcnn_mobilenet_v3_large_320_fpn,
    FasterRCNN_MobileNet_V3_Large_320_FPN_Weights,
)

weights = FasterRCNN_MobileNet_V3_Large_320_FPN_Weights.DEFAULT
model = fasterrcnn_mobilenet_v3_large_320_fpn(weights=weights).eval()
image = weights.transforms()(read_image("street.jpg")) # values in [0, 1]

with torch.inference_mode():
    prediction = model([image])[0]

boxes = prediction["boxes"]
labels = prediction["labels"]
scores = prediction["scores"]
```

Detection models receive a **list** of images, rather than one stacked tensor. Images may have different dimensions; the model handles its internal resizing and batching.

Image: https://pytorch.org/vision/stable/models.html

Fine-tuning: replace the detection head

Hochschule Karlsruhe University of Applied Sciences HKA

```python
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

model = fasterrcnn_mobilenet_v3_large_320_fpn(
    weights=FasterRCNN_MobileNet_V3_Large_320_FPN_Weights.DEFAULT
)
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(
    in_features, num_classes=2 # background + pedestrian
)
```

- Unlike Week 4 classification, the replacement head predicts a class *and* box refinements for each candidate region.
- Start by training the new head; optionally fine-tune later backbone layers.

Assignment 5 — Penn-Fudan pedestrian detection

Hochschule Karlsruhe University of Applied Sciences +HKA

- 170 street-scene images with 345 annotated pedestrian instances.
- Each image has an instance mask; derive one bounding box per pedestrian from its mask.
- One foreground class: `pedestrian`; background is implicit.

Why this dataset?

It is small enough for a lab, but has real images, multiple objects per image, variable image sizes, and ground truth from which boxes can be derived.

Dataset and workflow follow the official torchvision detection fine-tuning tutorial.

Assignment 5 — Core tasks

Hochschule Karlsruhe University of Applied Sciences HKA

1. Download and inspect Penn-Fudan: show image/mask pairs and report image and instance counts.
2. Implement a dataset that returns an image and target dictionary with `boxes`, `labels`, `image_id`, `area`, and `iscrowd`.
3. Visualise at least three ground-truth bounding boxes derived from masks.
4. Run COCO-pre-trained inference. Visualise predictions at score thresholds 0.3, 0.5, and 0.8; discuss the precision/recall trade-off.
5. Fine-tune the MobileNet Faster R-CNN head on a reproducible train/test split. Plot training loss and show detections on held-out images.
6. For at least five held-out images, calculate IoU between selected predictions and matched ground-truth boxes; discuss missed and duplicate detections.

Assignment 5 — Engineering details

Hochschule Karlsruhe
University of
Applied Sciences
HKA

Custom collation

```
def collate_fn(batch):
    return tuple(zip(*batch))

loader = DataLoader(dataset,
    batch_size=2,
    shuffle=True,
    collate_fn=collate_fn)
```

Each image has a different size and number of boxes, so ordinary tensor stacking is inappropriate.

Training call

```
model.train()
losses = model(images, targets)
loss = sum(losses.values())
loss.backward()
optimizer.step()
```

In training mode, torchvision returns several losses: proposal, classification, and box-regression losses. In evaluation mode, it returns detections.

Assignment 5 — Optional extensions

Hochschule Karlsruhe
University of
Applied Sciences
HKA

- Report COCO-style mAP with the torchvision reference evaluator and `pycocotools`.
- Compare frozen-backbone training with fine-tuning the last backbone layers.
- Compare the lightweight MobileNet detector with Faster R-CNN ResNet-50-FPN; include model size, training time, and qualitative results.
- Add horizontal-flip augmentation that transforms both image and boxes consistently.

Scope

Do not build a detector from scratch and do not annotate images manually. The learning objective is to understand detection targets, outputs, metrics, and transfer learning.

Takeaways

- Classification predicts one label for an image; detection predicts a set of labelled bounding boxes.
- Detection needs localisation-aware evaluation: IoU, precision, recall, and mAP.
- Faster R-CNN combines a CNN backbone, region proposals, and a detection head.
- Pre-trained COCO detectors transfer the Week 4 idea from image classification to structured visual prediction.

Happy coding!